

RTL Design Sherpa

Binary BCH Codec Hardware Architecture Specification 0.1 (draft)

October 3, 2026

Table of Contents

1 Bch Has Index.....	9
2 Document Information.....	10
2.1 Binary BCH Codec Hardware Architecture Specification.....	10
2.2 Revision History.....	11
2.3 Document Purpose.....	12
2.4 Intended Audience.....	13
2.5 Related Documents.....	14
3 Purpose and Scope.....	15
3.1 Purpose.....	15
3.2 Scope.....	16
3.3 Where this document stands.....	17
4 Document Conventions.....	18
4.1 Terminology.....	18
4.2 Naming.....	19
4.3 Notation.....	20
4.4 Decision markers.....	21
5 Definitions and Acronyms.....	22
6 Use Cases.....	25
6.1 Memory controller ECC.....	26
6.2 CCSDS telecommand link.....	27
6.3 DVB outer-code link.....	28
6.4 Standalone codec on a fabric port.....	29

6.5 Memory-to-memory job engine.....	30
7 Key Features.....	31
7.1 Decided (PRD section 3).....	31
7.2 Configurable, not yet chosen (chapter 6.2).....	32
7.3 Explicitly not features.....	33
8 System Context.....	34
8.1 An endpoint codec, not a mid-stream insert.....	34
8.1.1 Figure 2.1: Where the codec sits.....	34
8.2 The core inside a consumer.....	35
8.3 The standalone tops.....	36
8.4 Dependencies on the rest of the repository.....	37
9 Block Diagram.....	38
9.0.1 Figure 3.1: Codec block diagram.....	38
9.1 The encoder core.....	39
9.2 The decoder core.....	40
9.3 The GF layer.....	41
9.4 The adapters.....	42
9.5 Hierarchy.....	43
10 Data Flow.....	44
10.1 Encoder.....	44
10.2 Decoder.....	45
10.3 Block boundaries.....	46
10.4 Multiple bits per beat.....	47
11 Solver Options.....	48

12 Core Interface.....	50
12.1 Clock and reset.....	51
12.2 bch_encoder_core.....	52
12.3 bch_decoder_core.....	54
12.4 Timing rules.....	56
12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD).....	57
13 AXI-Stream Adapter.....	58
13.1 Parameters.....	59
13.2 Behaviour.....	60
14 AXI4 Job Adapter.....	61
14.1 Job interface.....	62
14.2 AXI4 master ports.....	63
14.3 Parameters.....	64
14.4 Behaviour.....	65
15 Register Block.....	66
16 Clock and Reset.....	68
16.1 Single clock.....	68
16.2 Reset.....	69
16.3 Clock gating.....	70
16.4 Frequency.....	71
17 Throughput.....	72
17.1 Sustained rate candidates (PRD D6).....	73
17.2 Measured placeholder.....	74
18 Latency.....	75

18.1 Encoder.....	76
18.2 Decoder.....	77
18.3 Buffer sizing.....	78
19 Resources.....	79
19.1 GF primitives per core.....	80
19.2 Storage.....	81
19.3 Order of magnitude.....	82
19.4 Scaling.....	83
20 System Requirements.....	84
20.1 The consumer provides.....	85
20.2 The consumer gets.....	86
20.3 Environment.....	87
21 Parameter Configuration.....	88
21.1 Code.....	89
21.2 Interface / build shape.....	91
21.3 Open decisions in one place.....	92
22 Candidate Profiles.....	94
23 Verification Strategy.....	96
23.1 Golden model.....	96
23.2 Test matrix.....	97
23.3 Test structure.....	99
23.4 Formal.....	100
23.5 Coverage.....	101
24 Synthesis and Implementation.....	102

24.1 Characterisation plan.....	102
24.2 Implementation notes.....	103

List of Figures

Figure 2.1: Where the codec sits.....	34
Figure 3.1: Codec block diagram.....	38

List of Tables

Table 1: Table 0.1: Related documents.....	14
Table 2: Table 1.1: Definitions and acronyms.....	22
Table 3: Table 2.1: Use cases and the profile and adapters each implies.....	30
Table 4: Table 2.2: Repository dependencies.....	37
Table 5: Table 3.1: Key-equation solver candidates.....	48
Table 6: Table 4.1: Clock and reset.....	51
Table 7: Table 4.2: Encoder core ports.....	52
Table 8: Table 4.3: Decoder core ports.....	54
Table 9: Table 4.4: AXI-Stream mapping (target).....	58
Table 10: Table 4.5: AXI-Stream adapter parameters.....	59
Table 11: Table 4.6: Job fields (target).....	62
Table 12: Table 4.7: AXI4 job adapter parameters.....	64
Table 13: Table 4.8: Register block outline (target).....	66
Table 14: Table 5.1: Throughput architecture candidates.....	73
Table 15: Table 5.1a: Measured cycles per block (placeholder).....	74
Table 16: Table 5.3: Encoder latency (placeholder).....	76
Table 17: Table 5.4: Decoder latency (placeholder).....	77
Table 18: Table 5.5: GF primitive counts (analytic placeholder).....	80
Table 19: Table 5.6: Storage (placeholder).....	81
Table 20: Table 6.1: Code parameters.....	89
Table 21: Table 6.2: Interface parameters.....	91
Table 22: Table 6.3: Open PRD decisions and their placeholders.....	92
Table 23: Table 6.4: Candidate profiles.....	94
Table 24: Table 6.5: Verification matrix.....	97
Table 25: Table 6.6: Characterisation steps.....	101

1 Bch Has Index

Generated: 2026-10-03

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

2 Document Information

2.1 Binary BCH Codec Hardware Architecture Specification

Property	Value
Document Title	Binary BCH Codec Hardware Architecture Specification
Version	0.1 (draft)
Date	October 3, 2026
Status	Draft – decisions D1-D6, D8, the D9 bits-per-beat sub-question, and D11 of the PRD are open; this document carries them as parameters
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.1	2026-10-03	RTL Design Sherpa	First draft, written before any RTL exists. Captures what the PRD has decided (D7: share the reed-solomon GF layer; D9 direction: valid/ready core with optional AXIS and AXI4 adapters; D10 direction: first consumer expected from a future memory-controller project; D12: scrambler out of scope) and carries the undecided items – field dimension m , correctable bits t , shortening, encoder/decoder mix, erasures, throughput architecture, bits per beat, generator conventions, key-equation solver – as parameters tied to their PRD decision IDs. Numbers are analytic placeholders; none are measured.

2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Binary BCH codec – what it is, where it sits, what it presents at its boundaries, what it costs and how it is configured. It is the document an integrator reads before instantiating the core in a memory controller, a compute engine or a link, and the document a verification engineer reads to know what must be proven.

It covers:

- the encoder and decoder cores and their valid/ready interfaces
- the optional AXI-Stream and AXI4 adapters for standalone use
- the configuration parameters and the candidate code profiles
- throughput, latency and resource estimates and how they scale
- integration requirements, verification strategy and synthesis plan

Micro-architecture – the GF arithmetic, the syndrome and Chien datapaths, the key-equation solver arrays, FSMs and signal-level timing – belongs to the companion Micro-Architecture Specification (MAS), which does not exist yet. Until it does, this HAS and the PRD are the block-level reference.

2.4 Intended Audience

- System architects deciding where error correction sits in a datapath
- Integrators instantiating the core or one of its adapters
- Verification engineers planning the golden-model comparison
- Anyone judging whether a binary BCH code, and which one, fits a channel

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Product Requirements	../.. /PRD.md	the decision table (D1-D12) and the candidate profiles
Architecture sketch	../bch_architecture_sketch.md	block diagram and reuse map (to be written)
References	../.. /References/README.md	the standards and papers, with source and licence
CLAUDE.md	../.. /CLAUDE.md	area facts for a session working here
Handbook	vault/handbook/design/	valid/ready contract, reset and clocking, filelists, generated-RTL discipline

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Purpose and Scope

3.1 Purpose

The Binary BCH codec adds bit-level forward error correction to a data path. An encoder appends parity bits to every block of k data bits; a decoder, given the $n = k + (n - k)$ received bits, corrects up to t bit errors anywhere in the block. The arithmetic is over $GF(2^m)$, but the code is binary: every error value is 1, so correction is a simple bit flip and no error-value computation is needed. A binary BCH code is the choice for channels whose errors are independent bit flips or multi-bit failures without a fixed symbol framing, such as raw flash pages and memory arrays.

The component exists so that a Sherpa consumer – a memory controller, a compute engine, a serial link – can drop a proven codec into its own datapath instead of building one. That is why the deliverable is a CORE with plain valid/ready streaming ports at both ends (PRD D9 direction): the consumer keeps its own interfaces and the codec is a block inside them.

3.2 Scope

In scope:

- `bch_encoder_core` and `bch_decoder_core`: systematic binary BCH(n , k) over $GF(2^m)$, parameterised in m , t and shortening
- the $GF(2^m)$ primitive layer imported from the reed-solomon component (PRD D7), used but not duplicated
- optional boundary adapters for standalone use: AXI-Stream at either end, or an AXI4 read or write engine with a job interface at either end, chosen independently per end
- the configuration registers, status and counters of the standalone tops
- verification against a software golden model, and FPGA characterisation

Out of scope (PRD section 6 and D12):

- soft-decision, list and folded BCH decoding
- concatenation with convolutional or LDPC codes, and interleaving beyond a bit-address permutation at the boundary
- any mid-stream insertion of the codec into a fabric (chapter 2.3 says why)
- a scrambler / randomizer inside the codec (D12: out unless a named standard consumer needs one)

3.3 Where this document stands

No RTL exists at revision 0.1. Everything here is derived from the PRD's decisions and the standards in References/. Where a number appears it is an analytic placeholder – a cell count from the algorithm, a latency bound from the pipeline – and is labelled so. Measured throughput, timing and resources arrive with the first synthesis run (chapter 6.5) and will replace the estimates in chapter 5.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 Document Conventions

4.1 Terminology

- **Bit:** one binary digit of the code word. The data and code streams are bit-oriented; there is no symbol framing inside the BCH block.
- **Symbol:** an m -bit field element used internally for $GF(2^m)$ arithmetic. `SYMBOL_WIDTH = m`. The syndrome and key-equation hardware work in symbols; the Chien search evaluates the locator at n bit positions.
- **Block:** n bits on the coded side, k bits on the data side. The core delimits blocks with a last flag on the final bit of each.
- **Beat:** one transfer on a valid/ready port; carries `BITS_PER_BEAT` bits of one codeword (PRD D9 TBD).
- **Profile:** one fully-resolved parameter set – m , t , n , k , first root, primitive polynomial – typically named after the standard it matches (chapter 6.3).
- **Core:** `bch_encoder_core` or `bch_decoder_core`, valid/ready at both ends.
- **Adapter:** the optional AXIS or AXI4 boundary around a core.

4.2 Naming

Module and signal names follow the repository conventions (`vault/handbook/design/`): `aclk` / `aresetn`, active-low reset through the `ALWAYS_FF_RST` / `RST_ASSERTED` macros, `r_` registers and `w_` wires internally, `valid/ready` per the house contract. Parameters are upper case. Generated files carry a “Generated by” banner and live in `rtl/generated/`.

4.3 Notation

- n, k, t, m as above; $n - k$ parity bits. The designed minimum distance is $2t + 1$.
- α : the primitive element of $GF(2^m)$ defined by the primitive polynomial $p(x)$; b : the first root of the generator polynomial $g(x) = \text{lcm}\{\text{minimal polynomials of } \alpha^b \dots \alpha^{b+2t-1}\}$ over $GF(2)$.
- $\Lambda(x)$: error-locator polynomial; S_i : syndromes.
- Counts quoted “for the reference profile” mean the CCSDS (63,56) profile, $t = 1$, $m = 6$, one bit per beat, unless a table says otherwise.

4.4 Decision markers

Where the PRD has decided something, this document states it as fact and cites the decision (for example “PRD D7”). Where the PRD has not, the text says **TBD** and names the decision; chapter 6.2 collects every TBD in one table.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

5 Definitions and Acronyms

Table 1.1: Definitions and acronyms

Term	Meaning
BCH(n, k)	Binary Bose-Chaudhuri-Hocquenghem code with n coded bits per block, k of them data
$GF(2^m)$	the Galois field with 2^m elements; internal symbols and all codec arithmetic live in it
m	field dimension; each internal symbol is m bits
t	the number of bit errors the code corrects per block
designed distance	$2t + 1$, the guaranteed minimum Hamming distance of the code
minimal polynomial	the monic irreducible polynomial over $GF(2)$ having α^i as a root in $GF(2^m)$; $g(x)$ is the lcm of the minimal polynomials of the consecutive roots $\alpha^b \dots \alpha^{(b+2t-1)}$
narrow-sense	a BCH code whose first root $b = 1$; non-narrow-sense has $b \neq 1$ (for example CCSDS uses $b = 112$ in its field representation)
primitive BCH	$n = 2^m - 1$, the full-length cyclic code
shortened BCH	$n < 2^m - 1$, derived from the full-length code by treating the missing leading bits as zero
evenness shortcut	in characteristic 2, $S_{2j} = S_j^2$, so only the t odd syndromes $S_1, S_3, \dots, S_{2t-1}$ are independent; the syndrome unit computes t GF values, not 2t
systematic	the k data bits pass through unchanged and the n - k parity bits

Term	Meaning
	follow them
syndrome	$S_i = R(\alpha^{(b+i)})$, the received polynomial evaluated at a generator root; all zero if and only if the block is a codeword
key equation	the relation the solver inverts to find $\Lambda(x)$ from the odd syndromes
riBM	reformulated inversionless Berlekamp-Massey, the default key-equation solver candidate (Sarwate and Shanbhag 2001)
Euclid / ME	the modified Euclidean algorithm in its inversionless systolic form (Shao et al. 1985), the alternative solver candidate
step-by-step decoding	Massey's 1965 procedure that corrects syndromes one error position at a time, attractive for small t
PGZ	Peterson-Gorenstein-Zierler direct matrix solve over the syndromes, another small- t candidate
Chien search	evaluating Λ at every one of the n bit positions to find the roots, hence the error locations
Forney stage	the RS decoder stage that computes error magnitudes; absent in a binary BCH decoder because every error value is 1
bit-level correction	the Chien search walks n bit positions and flips the bit wherever it finds a root; it does not evaluate symbol positions
erasure	a bit whose position is known to be unreliable (flagged at the input); not

Term	Meaning
	currently enabled (PRD D5 TBD)
AXIS	AXI4-Stream
PRD	Product Requirements Document (../PRD.md)
MAS	Micro-Architecture Specification (companion, not yet written)
FUB	functional unit block

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Use Cases

The codec is an endpoint block (chapter 2.3). Its use cases are the endpoints a Sherpa design has or may grow, each matched to one of the PRD's candidate profiles. None is yet a committed consumer – that is PRD decision D10 – so this chapter is the menu the first consumer chooses from.

6.1 Memory controller ECC

A memory controller encodes on its write path and decodes on its read path. Both cores sit inside the controller's own datapath behind its scheduler, which already delimits bursts and so provides the block boundary. The BCH code protects a page of bits plus the spare area that holds the parity; it is chosen from the device's raw bit-error rate and page geometry (the Cai survey and the Nabipour paper size this). This is the expected first consumer (PRD D10 direction) and the use case that argues most strongly for the bare valid/ready core.

6.2 CCSDS telecommand link

The CCSDS 231.0-B-4 standard specifies a (63,56) modified BCH code, $t = 1$, concatenated with a convolutional code inside the Command Link Transmission Unit. It is a small, fully worked example: the generator is given in figure 3-2 of the standard, the field is $GF(2^6)$, and the correction is a single bit anywhere in the 63-bit command frame. It is the natural first RTL profile for bring-up even if no consumer adopts it.

6.3 DVB outer-code link

DVB-T2, DVB-S2 and DVB-C2 use a binary BCH code as the outer stage of an LDPC concatenation. The BCH block cleans the LDPC error floor. This use case matters only if a broadcast consumer appears; the relevant standards are linked, not stored, in [References](#)/.

6.4 Standalone codec on a fabric port

The core wrapped in AXI-Stream at both ends, inserted where a stream already has block boundaries (TLAST) and room for the parity – a framer’s output, a DMA’s stream port. This is the AXIS/AXIS adapter pairing and the simplest way to bring the block up in a system.

6.5 Memory-to-memory job engine

The core wrapped in an AXI4 read engine and an AXI4 write engine, driven by jobs from its register block or a descriptor stream: encode this buffer into that one, or decode. A DMA with a transform. This is the AXI4/AXI4 pairing and the natural test vehicle for larger profiles.

Table 2.1: Use cases and the profile and adapters each implies

Use case	Profile	Adapter pairing	Erasures	Notes
Memory controller ECC	m = 13-16, t per page model	none (bare cores)	optional (D5 TBD)	the expected first consumer
CCSDS telecommand	(63,56), m = 6, t = 1	none, or AXIS/AXIS	no	natural first RTL profile
DVB outer-code link	m, t per standard	none, or AXIS/AXIS	no	only if a broadcast consumer appears
Standalone stream codec	any	AXIS/AXIS	optional	bring-up vehicle
Memory-to-memory job	any	AXI4/AXI4	optional	test vehicle for large profiles

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

7 Key Features

7.1 Decided (PRD section 3)

- **Valid/ready core at both ends** (D9 direction). bch_encoder_core and bch_decoder_core present the house streaming contract: valid, ready, data, a last flag on the final bit of a block; the decoder adds a per-block status out. A consumer instantiates the core and nothing else.
- **Optional adapters, selectable per end** (D9 direction). INTAKE_IF and OUTLET_IF each NONE (default), AXIS or AXI4, independently, so AXIS-in/AXI4-out and the other pairings are one core with different adapters generated. Only the adapters selected exist in the netlist.
- **GF primitives imported, not copied** (D7). gf_pkg, gf_mul, gf_mul_const and gf_inv are taken from the reed-solomon component's rtl/gf/ tree. Nothing BCH-specific lives in the RS tree.
- **Bits per beat is a new interface parameter** (D9 TBD). Unlike the RS codec there is no SYMBOLS_PER_BEAT; a beat carries a slice of one codeword. The exact width and the partial-beat mask are open sub-questions of D9.
- **No Forney stage** (structural fact of binary BCH). Every error value is 1, so the Chien search drives a direct bit flip. The error-value hardware that the RS decoder needs is absent here.
- **Evenness shortcut in the syndrome unit** (structural fact of binary BCH). Only the t odd syndromes $S_1, S_3, \dots, S_{2t-1}$ are computed; the even syndromes are obtained by squaring in $GF(2^m)$.
- **Generated field constants** (R3). The primitive polynomial, generator polynomial and log/antilog tables are emitted by a Python generator into rtl/generated/ from the profile parameters; no field constant is a literal in the datapath.
- **Every block reports** (R2). The decoder never silently passes a failed block: per-block status carries ok / corrected count / uncorrectable, and running counters feed the register block.
- **Golden-model verification** (R4). Every profile is checked against galois on random data, error patterns up to t and beyond t (to prove the uncorrectable flag), and erasures where enabled.

7.2 Configurable, not yet chosen (chapter 6.2)

- m , the field dimension, and t , the correctable bit count (D1, D2)
- shortening (D3) – full length or a shortened block
- whether both cores are built, or encoder-only / decoder-only (D4)
- erasure decoding present or absent (D5)
- throughput architecture and bits per beat (D6, D9)
- per-profile generator conventions: first root b , primitive polynomial (D8)
- key-equation solver algorithm (D11)
- the first consumer, which fixes all of the above (D10)

7.3 Explicitly not features

- Soft-decision or list decoding, interleaving beyond a boundary permutation, concatenation with other codes.
- A scrambler / randomizer inside the codec (D12: out unless a named standard consumer needs one).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 System Context

8.1 An endpoint codec, not a mid-stream insert

BCH protects a block, not a wire. The encoder must see k bits before it can emit the first parity bit, and the decoder must see all n bits before it can correct any of them, because the syndromes are a function of the whole block. The code therefore lives where a block boundary already exists and where the extra $n - k$ bits have somewhere to go. The expected consumer – a flash or memory controller – places it at an endpoint: encode on the write path as the page+spare image is assembled, decode (and correct, and report) on the read path before the data leaves. The parity rides the spare area; the correction belongs right after the channel that corrupts.

Dropping the codec into the middle of a fabric fails on three counts: the parity changes the bit count, so the downstream must run faster or the upstream must leave gaps; block alignment must survive every hop, which is why every BCH standard sits behind a sync marker, a fixed frame or a page boundary; and the correction belongs immediately after the channel that corrupts, not several repackings later.

8.1.1 Figure 2.1: Where the codec sits

Where the codec sits

Where the codec sits

Three endpoint shapes: a memory controller with an encoder on its write path and a decoder on its read path; a telecommand link pair with the BCH code inside the CLTU; and the standalone memory-to-memory job engine built from the AXI4 adapters.

8.2 The core inside a consumer

With valid/ready at both ends the core is a block in the consumer's own datapath. The consumer supplies block boundaries (its scheduler, framer or job control already knows where they are), absorbs the n/k rate change, and keeps its own external interfaces. Nothing in the consumer's protocol handling changes; what changes is that k bits go in and n come out (or n in, k out with a status).

8.3 The standalone tops

When there is no consumer datapath to sit inside – a codec on a fabric port, a memory-to-memory transform – the standalone tops wrap the core in adapters (chapters 4.2 and 4.3) and a register block (chapter 4.4), and the existing AXI-Stream and AXI4 monitors observe the boundaries onto the monitor bus.

8.4 Dependencies on the rest of the repository

Table 2.2: Repository dependencies

Dependency	Used for
projects/components/ecc-ip/reed-solomon/rtl/gf: gf_pkg, gf_mul, gf_mul_const, gf_inv	the shared GF(2 ^m) primitive layer (PRD D7)
rtl/amba/gaxi: gaxi_skid_buffer, gaxi_fifo_sync	stage decoupling; the decoder's block buffer
rtl/common: counter_bin, shifter_beat_pack, shifter_lfsr_galois	positions and iterations; bit packing; any boundary LFSR
rtl/amba/axis4: axis4_slave, axis4_master (+ _monlite)	AXIS adapters
rtl/amba/axi4: axi4_master_rd, axi4_master_wr	AXI4 adapters' timing wrappers
rtl/amba/monitor: axis_monitor_lite	boundary observation onto monbus
bin/peakrdl_generate.py + utility-ip/converters APB-to-cpuif path	the register block and its APB attachment
dma-ip/stream/rtl/fub: axi_read_engine, axi_write_engine	the model for the AXI4 job engines
galois, aff3ct (PyPI / standalone, MIT)	DV golden model and cross-check

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

9 Block Diagram

9.0.1 Figure 3.1: Codec block diagram

Codec block diagram

Codec block diagram

The two cores, the shared GF layer imported from the reed-solomon component, and the optional adapters that wrap a core for standalone use.

9.1 The encoder core

Four blocks in a straight line. `bit_unpack` (or its beat-to-bit serialiser) slices each beat into the `BITS_PER_BEAT` lanes and tracks the position in the block. `gf_lfsr_encoder` is the systematic encoder: a $2t$ -stage shift register over $GF(2^m)$ with the generator polynomial's coefficients as constant multipliers on its taps; it is cleared at block start, the k data bits shift through (the RS-shaped LFSR consumes them serially or in the parallel form chosen by `D6`), and it then holds the parity symbols. `parity_mux` passes the data bits and then appends the parity, raising `last` on the final one. `bit_pack` reassembles beats. A skid buffer after the unpack and before the pack decouples the stages.

9.2 The decoder core

The received block is written into `block_buffer` and, in the same cycles, into `syndrome_unit`, whose `t` cells each accumulate one odd syndrome by Horner's rule as the bits arrive. When the block is complete the odd syndromes go to `key_equation_solver` – one of the candidates from PRD D11 – which produces the error-locator polynomial in at most $2t$ iterations, or flags the block uncorrectable. `chien_search` then walks every one of the `n` bit positions, one per cycle (or more if D6 chooses a parallel tree), in lock-step with the buffer read-out; where it finds a root, `corrector` flips the bit leaving the buffer. `status` counts corrections and raises the per-block verdict with the final bit. An all-zero syndrome set bypasses the solver and drains the buffer uncorrected.

Because the code is binary there is no Forney evaluator: the error value is always 1. A second syndrome unit over the corrected stream may be added as a re-check, matching the RS decoder's release-on-verdict discipline; that is a D11 / implementation detail, not part of the locator search.

9.3 The GF layer

The GF layer is imported from `projects/components/ecc-ip/reed-solomon/rtl/gf/`. `gf_pkg` holds the field: the primitive polynomial, the generator coefficients and the log/antilog tables, all generated from the profile parameters. Three primitives are built on it: `gf_mul_const` (a constant multiply, an XOR network), `gf_mul` (full multiply, AND array plus reduction) and `gf_inv` (table inverse). Everything arithmetic in both cores is one of these three; the block counts are in chapter 5.3.

9.4 The adapters

Present only when `INTAKE_IF` or `OUTLET_IF` is not `NONE`. The AXI-Stream adapter is the house `axis4_slave` (intake) or `axis4_master` (outlet) timing wrapper with `TLAST` as the block boundary and `TUSER` carrying any erasure flags in or status out. The AXI4 adapter is a read engine (job in, stream out) or a write engine (stream in, bursts out) with a job controller that sequences them from the register block or a descriptor stream. Chapter 4 specifies each.

9.5 Hierarchy

The complete bottom-up hierarchy – every block, what it does and every component it instantiates with counts – will be recorded in a companion FUB catalog once the RTL exists. This chapter names the blocks; that catalog will be the authority on their contents.

[RTL Design Sherpa](#) · [Learning Hardware Design Through Practice](#) [GitHub](#) · [Documentation Index](#) · [MIT License](#)

10 Data Flow

10.1 Encoder

One beat in, one beat out, with the output stream (n/k) longer than the input. The exact shape depends on PRD D6 (throughput architecture) and the D9 bits-per-beat decision.

1. A beat of `BITS_PER_BEAT` data bits arrives with `in_valid`; the core accepts it when the encoder can take it (`in_ready`).
2. Each bit enters the LFSR (updating the parity registers) and is passed straight to the output mux – systematic encoding means the data are not delayed by the code, only by the pipeline.
3. On the k -th bit (`in_last`), the mux switches to the parity registers and emits the $n - k$ parity bits over the following cycles, then raises `out_last`; the LFSR clears for the next block.
4. While parity drains, `in_ready` is low: the encoder cannot start a new block until the previous block's parity has left. A consumer that wants full-rate back-to-back blocks provides an input FIFO of at least $n - k$ bits upstream, or accepts the gap.

10.2 Decoder

One beat in, one beat out, with a block of latency in between.

1. Received bits are written to the block buffer and fed to the syndrome unit. Erasure flags, if enabled, are recorded alongside each bit.
2. On `in_last` the `t` odd syndromes are complete. If all are zero the block is clean: the solver is skipped and the buffer drains through the corrector unchanged, `status = ok`.
3. Otherwise the key-equation solver runs. If the resulting locator degree exceeds `t`, or is zero, the block is already known to be uncorrectable.
4. Chien search steps through bit positions `0 .. n-1`, one (or more, per D6) per cycle, while the buffer reads out the same positions. At each root the corrector flips the bit.
5. On the last position the verdict is final: uncorrectable if the root count differs from the degree, or if a re-check over the corrected stream shows a non-zero syndrome. That last check catches blocks with more than `t` errors whose wrong locator still has the right number of roots.
6. The block is released only once its verdict exists. Corrections are applied on the way out, and not at all when the block is uncorrectable, so an uncorrectable block leaves exactly as it arrived. The status ports hold the verdict for every beat of the block and `out_last` marks its `k`-th bit.
7. The next block's bits arrive during steps 3-6 and the one after that is being solved while this one is walked; the buffers are sized for that overlap, so the decoder sustains one block per `n` cycles in the serial case.

10.3 Block boundaries

The core does not count bits to find a block: `in_last` is the boundary, and the core checks it. A block shorter or longer than the configured `n` (or `k` on the encoder side) is reported as a framing error in status and the core resynchronises on the next `in_last`. A shortened code ($N_BITS < 2^m - 1$) is handled by the core treating the missing leading bits as zero in the syndromes and by offsetting the Chien positions; the consumer sees only the shortened block.

10.4 Multiple bits per beat

With $\text{BITS_PER_BEAT} = B > 1$ every stage works a beat per cycle: the encoder LFSR advances by the beat's bit count, the syndrome cells take B Horner steps, and the Chien cells evaluate B positions from one set of registers, each lane a constant multiply away from the beat's first position. The solver works on the t odd syndromes and is unaffected. Blocks are still delimited by last; a per-beat keep mask is needed for partial final beats (the open D9 sub-question).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 Solver Options

The key-equation solver is the block whose algorithm is not yet chosen (PRD D11). Three candidates are attractive for a binary BCH decoder. They all invert the same key equation from the t odd syndromes to the error-locator polynomial $\Lambda(x)$. Because there is no Forney stage, the locator is the only polynomial the Chien search needs.

Table 3.1: Key-equation solver candidates

	inversionless BM ("RIBM")	modified Euclidean ("EUCLID")	step-by-step / PGZ ("SMALL_T")
Algorithm	reformulated inversionless Berlekamp-Massey, Sarwate and Shanbhag 2001; seeded with the t odd syndromes	Sugiyama's Euclidean algorithm with cross-multiplication in place of division (Shao et al. 1985)	Massey 1965 step-by-step correction, or the Peterson-Gorenstein-Zierler direct matrix solve for small t
Array	$3t + 1$ processing elements, 2 registers each	four register arrays: R and Q of $2t + 1$ symbols, the shifted locator pair of $2t + 3$	control-heavy but small datapath: one or a few GF multipliers and a syndrome update loop
GF multipliers	2 per element: $6t + 2$	2 per coefficient of each pair: $8t + 8$	$O(t^2)$ logic or a single iterative multiplier
Iterations	exactly $2t$	data-dependent, $t + 1 \dots 2t$	one per located error position, up to t
Critical path per iteration	one multiply and one XOR, no feedback across the array	cross-multiply feeding a degree comparison that drives the swap: longer	depends on implementation; often dominated by syndrome-update MUX tree
Control	discrepancy select and gamma	two nominal degree counters,	position counter, syndrome-test

	inversionless BM ("RIBM")	modified Euclidean ("EUCLID")	step-by-step / PGZ ("SMALL_T")
	update	the normalise / cross / swap rule	loop, error- counter update
Readability	dense; the reformulation is not obvious from the textbook	the textbook algorithm, recognisable step by step	the simplest conceptually for very small t
Best fit	general t; the default if no consumer names a small-t profile	a cross-check and a readable alternative	t <= 8 (flash pages often sit here); smallest area at tiny t

No default is selected at revision 0.1. The choice is D11. Two solvers agreeing as the cross-check, per the RS D11 philosophy, is the recommended verification posture: the golden model runs every candidate on every block and requires identical roots, and the RTL is built with whichever algorithm the first consumer's rate and area target prefers.

Nothing else in the decoder knows which solver produced Lambda. Syndromes, Chien, the block buffer and the corrector are the same modules in every candidate build.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 Core Interface

Both cores present the house valid/ready streaming contract (vault/handbook/design/valid-ready-contracts.md) at each end: valid and data are held stable until ready, ready may depend on valid, valid must not depend on ready. Widths are in bits; B is BITS_PER_BEAT (PRD D9 TBD).

12.1 Clock and reset

Table 4.1: Clock and reset

Port	Dir	Width	Description
aclk	in	1	one clock for the whole core
aresetn	in	1	active-low reset, per the component convention (chapter 4.5)

12.2 bch_encoder_core

Table 4.2: Encoder core ports

Port	Dir	Width	Description
in_valid	in	1	a beat of data bits is offered
in_ready	out	1	the core takes it this cycle
in_data	in	B	B data bits, bit 0 in the low bit TBD / D9
in_keep	in	B	which bits of the beat are present: low-aligned, all ones except possibly the final beat of a block TBD / D9
in_last	in	1	this beat carries the k-th data bit of the block
out_valid	out	1	a beat of coded bits is offered
out_ready	in	1	the consumer takes it
out_data	out	B	B coded bits: the data beats as received, then the n - k parity bits TBD / D9
out_keep	out	B	present-bit mask of the beat: partial at the end of the data and at the end of the parity

Port	Dir	Width	Description
			TBD / D9
out_last	out	1	this beat carries the n-th coded bit of the block
frame_err	out	1	pulse: a block ended with other than k data bits; it was still encoded as given

12.3 bch_decoder_core

Table 4.3: Decoder core ports

Port	Dir	Width	Description
in_valid / in_ready	in / out	1	as the encoder
in_data	in	B	B received bits TBD / D9
in_keep	in	B	present-bit mask TBD / D9
in_last	in	1	this beat carries the n-th received bit
in_erase	in	B	(only with ENABLE_ERASURES) bit is a known erasure TBD / D5
out_valid / out_ready	out / in	1	as the encoder
out_data	out	B	B corrected data bits; parity lanes are dropped, so the block is k bits TBD / D9
out_keep	out	B	present-bit mask, low-aligned, partial only on the block's final beat TBD / D9
out_last	out	1	this beat carries the k-th data bit of the block
out_status_ok	out	1	held for every beat of the block: it had no errors

Port	Dir	Width	Description
out_status_corrected	out	$\log_2(t)+1$	held for every beat of the block: bits corrected (0 .. t)
out_status_uncorrectable	out	1	held for every beat of the block: correction failed; the data are exactly as received
out_status_frame_err	out	1	held for every beat of the block: block length was not n; the block passes through uncorrected, its first length - (n - k) bits as data

12.4 Timing rules

- `in_last` is the block boundary. The core does not infer boundaries by counting; it checks the count against `n` (or `k`) and reports a mismatch.
- The encoder holds `in_ready` low while it drains the parity of the previous block. A consumer wanting no gap buffers that many beats upstream.
- Keep masks are low-aligned. A partial beat anywhere but a block's last is a framing error on the decoder and a count mismatch on the encoder.
- The decoder accepts the next block while it is still correcting the previous one, up to the depth of its block buffer; `in_ready` falls only when the buffer is full, which under sustained back-pressure on `out_ready` it will be.
- Status outputs describe the block being emitted and are valid on every beat of it, first to last; a consumer may sample them with `out_last` or with the first beat. A block is not released until its verdict is final, so an uncorrectable verdict is never preceded by altered bits.
- Reset clears block state and the parity and syndrome registers; the block buffer is not reset (its contents are indexed by pointers that are).

12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)

When enabled, `in_erase` marks bits the consumer already knows are bad (a known-bad page region). Each erasure costs one parity bit instead of two: the decoder corrects e errors and f erasures with $2e + f \leq 2t$. The exact mapping of a per-beat or per-job erasure bitmap to `in_erase` is part of the D5 / D9 adapter work. When disabled the port does not exist and the erasure-locator logic is not generated.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 AXI-Stream Adapter

Generated at an end when its *_IF parameter is "AXIS". It mirrors the posture of the reed-solomon AXI-Stream adapter (PRD D9 direction) but is not yet designed; the table below is the target contract **TBD / D9**.

The adapter is the house axis4_slave (intake) or axis4_master (outlet) timing wrapper with the core's stream mapped onto the AXI-Stream signals.

Table 4.4: AXI-Stream mapping (target)

AXIS signal	Core signal	Notes
TVALID / TREADY	valid / ready	one-to-one
TDATA[B-1:0]	data	B bits per beat
TKEEP[B/8-1:0] or per-bit	keep	byte keep or bit mask; the BCH-specific partial-beat encoding is part of D9
TLAST	last	the block boundary
TUSER (intake, decoder)	in_erase[B-1:0]	erasure flags, one per bit lane, only when ENABLE_ERASURES = 1; otherwise the consumer's TUSER passes through the intake unused
TUSER (outlet, decoder)	status	{frame_err, uncorrectable, corrected[...], ok}, valid with TLAST
TID, TDEST	—	passed through unchanged from intake to outlet when both ends are AXIS; tied off otherwise

13.1 Parameters

Table 4.5: AXI-Stream adapter parameters

Parameter	Default	Meaning
AXIS_SKID_DEPTH	2	skid depth in the wrapper (2..8)
AXIS_ID_WIDTH, AXIS_DEST_WIDTH	0	pass-through sideband widths
AXIS_MONITOR	0	1 selects the _monlite wrapper variant, which attaches axis_monitor_lite to the port and emits onto the monitor bus

13.2 Behaviour

The adapter adds no protocol semantics: TLAST is the block boundary the core requires, and the rate change is visible as more beats out than in on the encoder (or fewer on the decoder). A stream that arrives without TLAST at block boundaries cannot be decoded and is reported as a framing error on every block.

[RTL Design Sherpa](#) · [Learning Hardware Design Through Practice](#) [GitHub](#) · [Documentation Index](#) · [MIT License](#)

14 AXI4 Job Adapter

Generated at an end when its *_IF parameter is "AXI4". It mirrors the posture of the reed-solomon AXI4 job adapter (PRD D9 direction) but is not yet designed; the description below is the target contract **TBD / D9**.

The intake form is a read engine: a job names a source address and a byte count, the engine issues AXI4 read bursts and presents the returned data as the core's input stream, inserting last every n (decoder) or k (encoder) bits. The outlet form is a write engine: it drains the core's output stream into AXI4 write bursts at the job's destination address, one block at a time, and reports each block's status in its completion. A job controller sequences the two.

14.1 Job interface

Jobs arrive from the register block (chapter 4.4) – source, destination, count, kick – or, with JOB_IF = "DESC", from a small AXI-Stream descriptor port carrying the same fields for chained jobs.

Table 4.6: Job fields (target)

Field	Width	Meaning
src_addr	AXI_ADDR_WIDTH	read engine start address (intake = AXI4)
dst_addr	AXI_ADDR_WIDTH	write engine start address (outlet = AXI4)
byte_count	32	bytes to read; must be a whole number of blocks
kick / done	1 / 1	start; completion pulse with the job's status summary
cfg_erasure	N_BITS	decoder only, ENABLE_ERASURES = 1: job-level erasure bitmap, sampled at cfg_start and sliced per beat by the same beat index that builds keep – the shape a known-bad column has TBD / D5

14.2 AXI4 master ports

Each engine presents one AXI4 master (read-only or write-only) through the house `axi4_master_rd / axi4_master_wr` timing wrapper: `AXI_ID_WIDTH`, `AXI_ADDR_WIDTH`, `AXI_DATA_WIDTH` (= `BITS_PER_BEAT` rounded up to a byte multiple), incrementing bursts up to `cfg_xfer_beats`, no narrow transfers. Responses other than OKAY are counted and reported in the job's done status; the data are still passed.

14.3 Parameters

Table 4.7: AXI4 job adapter parameters

Parameter	Default	Meaning
AXI_ADDR_WIDTH	32	address width
AXI_ID_WIDTH	4	ID width; the engines use one ID
AXI_MAX_BURST_BEATS	16	burst cap (cfg_xfer_beats may lower it at run time)
JOB_IF	"REGS"	"REGS" or "DESC"
AXI_MONITOR	0	1 attaches the _monlite wrapper variant

14.4 Behaviour

The read engine is modelled on STREAM's `axi_read_engine` and the write engine on its `axi_write_engine` – job valid/address/count in, done strobe out, a burst cap – reduced from STREAM's multi-channel, SRAM-coupled form to one job and a FIFO. Whether that is a reduction of STREAM's modules or a fresh instantiation is decided when the adapter is built. The job controller hands the write engine each block's destination before the block's first bit reaches it, so a decoder job's output blocks land contiguously with the parity removed.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 Register Block

The BCH component has no standalone tops and therefore no register block at revision 0.1. The bare cores are parameterised; their status is the per-block out_status port. When the AXI4 job adapter is built a register block will be added to the standalone top, attached through the converters' APB-to-cpuif path exactly as the reed-solomon standalone tops do.

The target register map is the reed-solomon map adapted to bit-level status:

Table 4.8: Register block outline (target)

Register	Access	Contents
ID	RO	component ID, version, KES_ALGO, ENABLE_* build flags
PROFILE	RO	m, t, n, k, first root b, primitive polynomial index of the build
CTRL	RW	enable, soft reset, erasure enable (if built)
JOB_SRC, JOB_DST, JOB_LEN	RW	job fields (AXI4 ends only)
JOB_KICK	WO	start the job
JOB_STATUS	RO	busy, done, last job's status summary, AXI response errors
STAT_BLOCKS	RO	blocks processed
STAT_CORRECTED_BLOCKS	RO	blocks with at least one correction
STAT_CORRECTED_BITS	RO	total bits corrected
STAT_UNCORRECTABLE	RO	blocks flagged uncorrectable
STAT_FRAME_ERR	RO	blocks with a length mismatch
IRQ_EN, IRQ_STATUS	RW / W1C	interrupt on uncorrectable, on job done, on frame error

Registers will be accessed by name through the generated regmap in DV and host code, never by offset (`vault/handbook/dv/registers-by-name.md`). Counters will be 32-bit, saturating, cleared by `CTRL.soft_reset`.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Clock and Reset

16.1 Single clock

The core and its adapters run on one clock, `aclk`. A consumer whose data path runs on another clock crosses at the consumer's boundary with the repository's CDC blocks (`rtl/cdc`); the core does not contain a crossing. For a standalone top whose AXI4 memory side runs on a different clock than the stream side, the adapter's timing wrapper is the place to insert the `axi4_cdc_{rd,wr}` pair, as the bridge does for its CDC slave ports; that is a build option for the adapter, not a property of the core.

16.2 Reset

`aresetn` is active-low and synchronous, per the component target convention. It is applied through the `ALWAYS_FF_RST` macros with `RST_ASSERTED` so the polarity follows the repository build (`vault/handbook/design/reset-and-clocking.md`). Reset clears control state, the parity and syndrome registers, position and iteration counters and the status. The block buffer's storage is not reset; its pointers are, so stale contents are never read.

16.3 Clock gating

None inside the core. The `_cg` variants of the AXIS and AXI4 wrappers may be selected at the adapters for a standalone top that wants boundary clock gating, following the rest of the repository.

16.4 Frequency

The decoder's clock is set by the key-equation solver's per-iteration critical path, then by the syndrome and Chien cells (one constant multiply plus an XOR). The exact paths are not yet characterised; chapter 5.3 gives the characterisation plan that turns this into a number per part.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

17 Throughput

All throughput numbers are **TBD**. The PRD requires (R5) that no number be claimed before it is measured in an out-of-context synthesis fixture per the repository's practice.

17.1 Sustained rate candidates (PRD D6)

The syndrome pass is the throughput cost: n bit positions are accumulated against t GF MACs. Three architectures are candidates.

Table 5.1: Throughput architecture candidates

Architecture	Bits per cycle	Cost	Best fit
Bit-serial syndrome pass	1	t parallel GF MACs, one per odd syndrome; Chien walks one position per cycle	smallest area; low clock-rate targets
Page-parallel unrolled syndrome tree	n / latency	unrolled GF reduction tree over the whole block; large combinational or deeply pipelined	highest throughput; large area
Symbol-parallel beats with per-beat serial bits	B per beat	B -bit input slice, each bit serially updates the t MACs; Chien evaluates B positions per cycle	balance; matches a byte/word memory datapath

The rate contract – the relationship between the consumer’s clock, the block size n , the bits per beat B , and the sustained blocks per second – must be written down once D6 decides. Until then the only honest statement is that the core consumes one beat per cycle when not back-pressured, and the number of bits per beat is itself open (PRD D9).

17.2 Measured placeholder

No synthesis or simulation fixture exists. The table below will be replaced by measured cycles per block after the first RTL and the out-of-context fixture run.

Table 5.1a: Measured cycles per block (placeholder)

Profile	Architecture	Bits/cycle	Cycles per block
CCSDS (63,56)	TBD	TBD	TBD
NAND flash shortened	TBD	TBD	TBD
DVB-T2/S2/C2	TBD	TBD	TBD

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

18 Latency

All latency numbers are **TBD**. The exact values depend on PRD D6 (throughput architecture), D9 (bits per beat), D11 (solver algorithm), and the pipeline depth chosen during micro-architecture.

18.1 Encoder

Table 5.3: Encoder latency (placeholder)

From	To	Cycles
first data bit in	first data bit out	pipeline only
k-th data bit in	first parity bit out	pipeline + 1
k-th data bit in	last parity bit out	pipeline + (n - k) / B

18.2 Decoder

Table 5.4: Decoder latency (placeholder)

Phase	Cycles	Notes
receive the block	n / B	t odd syndromes complete with the last bit
key equation	$\leq 2t$ (BM) or data-dependent (Euclid / small- t)	skipped when the syndromes are all zero
Chien walk + bit flip + re-check	n / B	one or more positions per cycle
drain	k / B	the data bits leave while the next block is walked
pipeline	TBD	descriptor skids, solver handoff, verdict stage, output FIFO
first bit in to first bit out	TBD	dominated by $2n/B$ plus solver and pipeline
first bit in to last bit out	TBD	dominated by $2n/B + k/B$ plus solver and pipeline

For a shortened code the n above is the shortened length.

18.3 Buffer sizing

The block FIFO must hold the current block while the next arrives during the Chien walk. Its depth is TBD and will be sized once D6 and D9 are fixed. The output FIFO holds a block's data bits while it waits for its verdict and the next block's while that one is walked. The release-on-verdict rule costs this second buffer and extra first-bit latency; it is the price of never emitting an altered bit on an uncorrectable block.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 Resources

All resource numbers are **TBD**. The PRD requires (R5) that no number be claimed before it is measured in an out-of-context synthesis fixture per the repository's practice. Chapter 6.5 is the plan that replaces this table with measurements.

19.1 GF primitives per core

The BCH core reuses the reed-solomon GF primitives (gf_mul_const, gf_mul, gf_inv) without duplicating them. The counts below are the expected structural forms once the RTL exists; they are not measured.

Table 5.5: GF primitive counts (analytic placeholder)

Primitive	Encoder	Decoder	Formula (target)
gf_mul_const	2t	2t + t	generator taps + syndrome constant multiplies
gf_mul	0	solver-dependent	BM: 6t + 2; Euclid: 8t + 8; small-t: $O(t^2)$ or fewer
gf_inv	0	0 or 1	binary BCH needs no Forney inverse; a small-t solver may use one
GF symbol registers	2t	t + solver array	encoder 2t parity; decoder t odd syndromes + locator

19.2 Storage

Table 5.6: Storage (placeholder)

Item	Size	Formula
block FIFO	TBD	$(n + \text{latency, rounded up}) \times B$ bits, plus erasure bit if enabled
output FIFO	TBD	$(2k + \text{latency, rounded up}) \times (B + 1)$: corrected bit, hit, last
re-check syndromes	t cells	a second syndrome_unit over the corrected stream
log / antilog tables (gf_inv)	$2 \times 2^m \times m$	LUT ROM at small m , BRAM or Itoh-Tsujii at large m
skid buffers	2 (encoder), 3 (decoder), 2 deep	stage boundaries

19.3 Order of magnitude

A `gf_mul_const` at $m = 8$ is at most 8 XOR trees of up to 8 inputs; a `gf_mul` is an $m \times m$ AND array with reduction. The total arithmetic cost scales with t and m , and with the D6 throughput choice. The first out-of-context run will say where the BCH decoder lands on the target parts.

19.4 Scaling

- m grows: every multiplier grows from $m \times m$, and the log/antilog tables scale as $2^m \times m$.
- t grows: syndrome cells, solver array and Chien lanes all grow linearly with t .
- Shortening: n is smaller than $2^m - 1$, so the Chien walk and buffers are shorter but the arithmetic width is unchanged.
- Bits per beat B (D9): syndrome and Chien datapaths scale with B ; the solver is unaffected.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

20 System Requirements

What a consumer must provide, and what it gets.

20.1 The consumer provides

- **Block boundaries.** `in_last` on the k -th data bit (encoder) or n -th coded bit (decoder) of every block. The consumer's scheduler, framer or job control already knows where blocks end; the core checks and reports, it does not infer.
- **Room for the rate change.** The encoder emits n bits for every k it takes; the decoder emits k for every n . The consumer's downstream (encoder) or upstream (decoder) absorbs the difference, or provides a FIFO of $n - k$ bits so the encoder's parity drain does not stall its input.
- **A single clock** and the repository reset convention (chapter 4.5).
- **Erasur knowledge**, if `ENABLE_ERASURES`: which bits are known bad, per bit or per job, depending on the adapter.
- **Configuration at build time.** The profile is parameters; nothing about the code is set at run time in the bare core.

20.2 The consumer gets

- Systematic encoding: data pass through unchanged and in order, parity follows.
- A per-block verdict on the decoder: ok, corrected (with count), uncorrectable, framing error – valid with the last bit of the block, so the consumer can act on the block it has just received.
- Data as received on an uncorrectable block, never silently altered.
- Determinism: no run-time state survives a block except the counters; the same input block yields the same output block every time.

20.3 Environment

- SystemVerilog-2012, synthesisable, no assertions in RTL (properties live in `formal/`), lint-clean under Verilator per the repository gate.
- Filelists under `rtl/filelists/`, registered in `bin/filelists.toml`, one per top, from the first module.
- Python golden model dependencies (`galois`, `aff3ct`) are DV-only.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 Parameter Configuration

Every parameter of the two cores and the standalone tops. **TBD** marks a PRD decision not yet taken; the value shown is a placeholder, not a default.

21.1 Code

Table 6.1: Code parameters

Parameter	Type	Range	Default	Meaning	PRD
FIELD_DIM	int	3..16	TBD	m; the field is $GF(2^m)$	D1
T_BITS	int	$1..(2^m-1)/2$	TBD	correctable bit errors per block	D2
N_BITS	int	$2t+1 .. 2^m-1$	TBD	block length in bits; $< 2^m-1$ is a shortened code	D3
PRIM_POLY	int	irreducible of degree m	TBD	primitive polynomial ; selects the field representation	D8
FIRST_ROOT	int	$0 .. 2^m-2$	TBD	b, first root of g(x); narrow-sense when b = 1	D8
ENABLE_ERASURES	bit		0 TBD	erasure input and erasure-locator logic (decoder)	D5
KES_ALGO	string	"RIBM", "EUCLID", "SMALL_T"	TBD	key-equation solver	D11

Derived: $K_BITS = N_BITS - DEGREE_G$, where $DEGREE_G$ is the degree of the generator polynomial computed by `gf_pkg` from `m`, `t`, `PRIM_POLY` and `FIRST_ROOT`; elaboration fails if K_BITS is below 1.

Every code parameter is fixed at elaboration (PRD D1-D3). There is no run-time `t` or `n`: the arrays are generate loops sized by `t`, the generator polynomial is computed by `gf_pkg` when the design elaborates, and a block format change is a rebuild.

21.2 Interface / build shape

Table 6.2: Interface parameters

Parameter	Type	Range	Default	Meaning	PRD
BITS_PER_BEAT	int	1 .. k	TBD	number of bits per valid/ready beat; the bus is a slice of one codeword	D9
BUILD_ENCODER	bit		1 TBD	include the encoder core	D4
BUILD_DECODER	bit		1 TBD	include the decoder core	D4
INTAKE_IF	string	"NONE", "AXIS", "AXI4"	"NONE"	intake adapter (standalone tops)	D9
OUTLET_IF	string	"NONE", "AXIS", "AXI4"	"NONE"	outlet adapter	D9

Adapter-specific parameters are in tables 4.5 and 4.7.

21.3 Open decisions in one place

Table 6.3: Open PRD decisions and their placeholders

Decision	What fixes it	Effect if left at the placeholder
D1 field dimension m	the first consumer's page or frame size	GF cell width, table size, Chien length are unresolved
D2 correctable bits t	the consumer's error model	parity count, solver size, Chien width are unresolved
D3 shortening	the consumer's block length	zero-fill on encode, position offset on decode are unresolved
D4 encoder/decoder mix	transmit-only, receive-only, or both	both cores are built; a consumer instantiates what it needs
D5 erasures	storage / RAID consumer says yes	no erasure port, smaller decoder
D6 throughput architecture	the consumer's clock and rate	the whole microarchitecture is unresolved
D8 generator conventions	the standard	field representation and generator taps are unresolved
D9 bits per beat and partial-beat mask	the consumer's datapath width	core port widths and adapter mapping are unresolved
D11 key-equation solver	area / latency target	solver cell counts and cycles are unresolved
D10 first consumer	you	none of the above changes until it appears

22 Candidate Profiles

A profile is a resolved parameter set. These three come from the standards and papers in References/; the first consumer picks one or defines its own.

Table 6.4: Candidate profiles

Profile	m	t	n / k	PRIM_PO LY	FIRST_R OOT	Source	Notes
CCSDS telecom mand	6	1	63 / 56	see figure 3- 2 of CCSDS 231.0-B- 4 section 3	112 in the CCSDS field represe ntation	CCSDS 231.0-B- 4 section 3 (stored in Referen ces/)	the classic comma nd link: small t, but a fully worked generat or in a free standar d – the natural first RTL profile
NAND flash ECC	13-16	12-72 class	page- plus- spare geometr y, shorten ed by constru ction	per controll er vendor	per controll er vendor	Cai et al. 2017 (stored) for the error landscap e; Nabipo ur & Javidan 2023 (stored) for (m,t)	the profile the memor y- controll er consum er is expecte d to name; n runs to

Profile	m	t	n / k	PRIM_PO LY	FIRST_R OOT	Source	Notes
						selection	thousands of bits, so D6 and the Chien length dominate
DVB-T2 / S2 / C2	per standard	per standard	per standard	per standard	per standard	ETSI EN 302 755 / 302 307 / 302 769 (linked in References/)	BCH cleans the LDPC error floor; interesting only if a broadcast consumer ever appears

The CCSDS row is the natural first RTL profile because the generator is explicit in the stored standard. The NAND flash row is the expected consumer but its exact (m, t, n) come from the controller vendor. The DVB row is linked, not stored, because ETSI restricts redistribution.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

23 Verification Strategy

23.1 Golden model

Every configuration is verified against a software binary BCH implementation on the same blocks: `galois` (MIT) is the first candidate because it exposes `galois.BCH` with explicit primitive-polynomial, generator and shortening control. `aff3ct` (MIT) is the cross-check. This is the model-first order from the reed-solomon TASK-002 lesson: the golden model is validated before any RTL is compared against it.

The testbench encodes with the model and checks the RTL encoder bit for bit; it injects errors into model-encoded blocks and checks the RTL decoder's corrected output and verdict against the model's decode.

23.2 Test matrix

Table 6.5: Verification matrix

Axis	Values	Why
profile	each row of table 6.4 the build admits	field constants and shortening differ per profile
errors per block	0, 1, t-1, t, t+1, random	0 exercises the bypass; t+1 and more must match the reference decoder's verdict – uncorrectable, or the identical miscorrection where the code itself cannot tell – and an uncorrectable block's data must be exactly as received
reference model	galois.BCH	the hardware algorithms in Python on the same field, validated against AFF3CT
erasures (if built)	0 .. 2t, mixed with errors up to $2e + f = 2t$	the erasure budget
bits per beat solver	1, and the consumer's B BM, Euclid, small-t where applicable	the B-fold datapaths identical roots on every corrected block
adapters	NONE/NONE first; then AXIS/AXIS and AXI4/AXI4; the mixed pairs follow	the core is proven before any wrapper
back-pressure	none, random on both ends, sustained on the output	the valid/ready contract and the block buffer's full behaviour
framing	short block, long block, missing last	the framing-error path

23.3 Test structure

cocotb tests under `dv/tests/` with TB classes in `dv/tbclasses/`, Pattern B (`cocotb_test_*` prefixes), the three run levels through both the `REG_LEVEL` grid and `TEST_LEVEL` depth, per the `test-patterns` skill. Traffic at the adapters uses the existing AXI-Stream and AXI4 BFM; nothing drives a valid/ready port by hand. Registers are read by name through the generated regmap.

23.4 Formal

No assertions in RTL. Candidate properties for formal/ blocks: the valid/ready contract at both ends of each core; the encoder's parity equals the golden LFSR for one block (bounded); the evenness shortcut $S_{2j} = S_j^2$ on the syndrome unit; the equivalence of the solver candidates on symbolic odd syndromes for small t ; the no-Forney structural fact that correction is a pure bit flip.

23.5 Coverage

Functional coverage on the matrix above plus: every odd syndrome cell non-zero at least once, every Chien position rooted at least once across the run, the uncorrectable path taken by both the degree check and the root-count check, and the re-check syndrome path when implemented.

[RTL Design Sherpa](#) · [Learning Hardware Design Through Practice](#) [GitHub](#) · [Documentation Index](#) · [MIT License](#)

24 Synthesis and Implementation

24.1 Characterisation plan

Numbers in chapter 5 are placeholders. They are replaced by measurements the way the repository measures every other block: an out-of-context synthesis fixture per configuration, on the same two parts the monitor and bridge characterisations use.

Table 6.6: Characterisation steps

Step	Fixture	Output
1	bch_encoder_core and bch_decoder_core, CCSDS (63,56) profile, B = 1	LUTs, flops, BRAM, WNS; the calibration point
2	decoder with each KES_ALGO candidate that is built	the solver comparison in table 3.1 measured
3	B = 8 and B = 64	the bits-per-beat scaling measured once D9 is decided
4	NAND flash shortened profile	large-m, large-t scaling
5	standalone tops with each adapter pair	adapter cost; the monitors' cost is already known

The fixture follows `projects/components/fabric-gen-ip/bridge/fpga/` (`make synth BRIDGE=<fixture>` on `make/fpga_flow.mk`) and reports through the same `summary.csv` shape, so the numbers land beside the bridge's and the monitor's.

24.2 Implementation notes

- All arithmetic is combinational XOR/AND networks; there are no carries, so the multiplier trees of rtl/math do not apply and the synthesiser's DSP inference is irrelevant.
- `gf_inv` at small `m` is two 2^m -entry ROMs and infers to LUTs; at large `m` it becomes BRAM or Itoh-Tsujii.
- The block buffer infers BRAM once the depth passes the LUT-RAM threshold; `REGISTERED = 1` on the FIFO adds a cycle and removes the read mux from the corrector's path if timing needs it.
- The riBM array has no cross-array feedback, so its cells may be pipelined per iteration if the clock demands; the Euclid array's degree compare cannot be pipelined without the degree-computationless variant.
- No clock gating in the core; boundary gating comes from the `_cg` wrapper variants at the adapters.